

کامپایلر NNGraph DSL

یک زبان دامنه‌محور برای توصیف معماری شبکه‌های عصبی
و تولید خودکار کد PyTorch

گزارش فنی پروژه

۲۶ خرداد ۱۴۰۵

فهرست مطالب

۳	چکیده
۴	۱ مقدمه
۴	۱.۱ انگیزه
۵	۲.۱ هدف
۵	۳.۱ دامنه کاربرد
۶	۲ کارهای مرتبط
۶	۱.۲ چارچوب NADER
۶	۲.۲ FANTLR
۶	۱.۲.۲ الگوی Listener در برابر Visitor
۷	۳.۲ پروژه EVM ANT LR
۸	۳ طراحی زبان
۸	۱.۳ ساختار برنامه
۹	۲.۳ گرامر
۱۰	۳.۳ سیستم انواع
۱۰	۴.۳ انواع لایه
۱۰	۱.۴.۳ لایه‌های پارامتردار
۱۱	۲.۴.۳ فعال‌سازی‌ها
۱۱	۳.۴.۳ عملیات گراف (بدون ماژول)
۱۲	۵.۳ نحو یال
۱۲	۶.۳ مثال کامل: MLP
۱۵	۴ پیاده‌سازی
۱۵	۱.۴ معماری کامپایلر
۱۶	۱.۱.۴ مسئولیت فایل‌ها
۱۶	۲.۴ تحلیل‌گر معنایی

۱۶	انتشار مقدار در درخت	۱.۲.۴
۱۷	قوانین اعتبارسنجی	۲.۲.۴
۱۷	قالب پیام خطا	۳.۲.۴
۱۷	FalseCtx_ برای خطاهای پس از پیمایش	۴.۲.۴
۱۸	مولد کد	۳.۴
۱۸	الگوریتم تخصیص متغیر	۴.۴
۱۹	ردیابی MLP (زنجیره خطی)	۱.۴.۴
۱۹	ردیابی ResBlock (فناوت سپس ادغام)	۲.۴.۴
۲۰	ردیابی InceptionBlock (دو شاخه موازی)	۳.۴.۴
۲۱	ردیابی TransformerEncoder (دو residual ضمنی)	۴.۴.۴
۲۲	موارد خاص تولید کد	۵.۴.۴
۲۲	ساختار کد خروجی	۶.۴.۴

۵ ارزیابی

۲۴	مجموعه آزمایش	۱.۵
۲۵	نتایج pass forward	۲.۵
۲۵	اثبات درستی متغیرها	۳.۵
۲۶	آمار کدبیس	۴.۵

۶ تصمیمات طراحی

۲۷	Listener در برابر Visitor	۱.۶
۲۷	load_from_listener() در برابر generate(traversal)	۲.۶
۲۷	نام‌گذاری متغیر با node_id	۳.۶

۷ نتیجه‌گیری و کارهای آینده

۲۸	نتیجه‌گیری	۱.۷
۲۸	کارهای آینده	۲.۷

۲۹ منابع

چکیده

NNGraph DSL یک زبان دامنه‌محور است که برای توصیف معماری شبکه‌های عصبی به صورت گراف جهت‌دار طراحی شده است. فایل‌های `nng` را به عنوان ورودی دریافت کرده و کد `PyTorch` اجرایی تولید می‌کند که یک زیرکلاس `nn.Module` کامل با متدهای `__init__` و `forward` است.

کامپایلر با `ANTLR4` نسخه `۲.۱۳.۴` ساخته شده و از الگوی `Listener` برای تحلیل معنایی و الگوریتم مرتب‌سازی توپولوژیک `Kahn` هم در تحلیل‌گر معنایی (تشخیص دور) و هم در مولد کد (ترتیب اجرا) استفاده می‌کند.

مجموعه آزمایش شامل ۵۵ تست در چهار فایل است. چهار معماری نمونه—`MLP`، `ResBlock`، `InceptionBlock` و `TransformerEncoder`—با موفقیت کامپایل شده و `forward pass` آن‌ها با شکل خروجی صحیح اجرا شده است.

فصل ۱

مقدمه

۱.۱ انگیزه

نوشتن زیرکلاس‌های `nn.Module` در PyTorch به صورت دستی برای معماری‌های پیچیده، تکراری و مستعد خطاست. برای یک شبکه با ده لایه، برنامه‌نویس باید:

- در `__init__` هر لایه را جداگانه با پارامترهای صحیح تعریف کند
 - در `forward` ترتیب اجرا را به صورت دستی مدیریت کند
 - در توپولوژی‌های شاخه‌دار، متغیرهای میانی را نام‌گذاری کند
 - مطمئن شود هیچ تانسور مشترکی پیش از استفاده بازنویسی نمی‌شود
- کد PyTorch زیر را در نظر بگیرید که باید به صورت دستی نوشته شود:

قطعه کد ۱.۱.۱ > نمونه boilerplate دستی برای یک ResBlock

```
1 class ResBlock(nn.Module):
2     def __init__(self):
3         super(ResBlock, self).__init__()
4         self.conv1 = nn.Conv2d(
5             in_channels=64, out_channels=64,
6             kernel_size=3, stride=1, padding=1)
7         self.bn1 = nn.BatchNorm2d(num_features=64)
8         self.relu1 = nn.ReLU()
9         # ... and so on for each layer
```

```
10
11     def forward(self, x):
12         identity = x
13         x = self.conv1(x)
14         x = self.bn1(x)
15         x = self.relu1(x)
16         # ... manage residual connection manually
17         x = x + identity
18         return x
```

با NNGraph DSL، معماری یک‌بار به صورت گراف توصیف می‌شود و کامپایلر این کد را تولید می‌کند.

۲.۱ هدف

هدف NNGraph DSL این است که فایل `nng`. تنها منبع حقیقت معماری باشد. برنامه‌نویس گراف را توصیف می‌کند؛ کامپایلر تمام boilerplate را تولید می‌کند.

۳.۱ دامنه کاربرد

این ابزار برای توصیف معماری‌های ثابت مناسب است—شبکه‌هایی که گراف محاسباتی آن‌ها در زمان تعریف مدل مشخص است. پشتیبانی از کنترل جریان پویا درون `forward` در نسخه‌های آینده برنامه‌ریزی شده است.

فصل ۲

کارهای مرتبط

۱.۲ چارچوب NADER

تعریف ۱.۱.۲ NADER

NADER (Neural Architecture Design via Representation) چارچوبی است که معماری شبکه‌های عصبی را به صورت گراف محاسباتی جهت‌دار (DAG) مدل می‌کند. هر گره یک عملیات لایه است و هر یال جریان تنسور را نشان می‌دهد.

NNGraph DSL این مدل گراف را به عنوان پایه طراحی زبان به کار می‌گیرد.

۲.۲ ANTLR

تعریف ۱.۲.۲ ANTLR

ANTLR (ANOther Tool for Language Recognition) ابزاری است که از یک فایل گرامر g4. به صورت خودکار lexer، parser و listener/visitor تولید می‌کند. نسخه مورد استفاده: ANTLR ۴.۱۳.۲.

۱.۲.۲ الگوی Listener در برابر Visitor

ANTLR^۴ دو الگو برای پیمایش درخت تجزیه ارائه می‌دهد:

• **Listener**: ParseTreeWalker به صورت خودکار درخت را پیمایش می‌کند و متدهای `enter*/exit*`

را فراخوانی می‌کند. هیچ مقدار بازگشتی نیاز نیست.

• **Visitor**: برنامه‌نویس پیمایش را کنترل می‌کند و هر متد یک مقدار برمی‌گرداند.

این کامپایلر از الگوی Listener استفاده می‌کند. متدهای *exit در custom_listener.py حالت را در self.nodes و self.edges انباشته می‌کنند—این رویکرد برای ساختن مدل گراف از درخت تجزیه طبیعی‌تر است.

۳.۲ پروژه ANTLR EVM

این کامپایلر سبک و ساختار کد را از پروژه EVM موجود در PycharmProjects/Antlr به ارث می‌برد. سه فایل از آن پروژه مستقیماً استفاده می‌شوند:

• ast.py – کلاس AST و TreeNode

• make_ast_subtree.py – ترکیب گره‌های درخت تجزیه

• ast_to_networkx_graph.py – تجسم اختیاری گراف

فصل ۳

طراحی زبان

۱.۳ ساختار برنامه

هر فایل nng . از سه بلوک تشکیل شده است. بلوک config اختیاری است:

قطعه کد ۱.۱.۳ > ساختار کلی یک فایل nng.

```
1 model <Name> {
2     input  <id> : tensor(<shape>)
3     output <id>
4 }
5
6 graph {
7     node <id> : <LayerType>(<params>)
8     ...
9     edge <src> -> <dst>
10    edge <src> -> <dst> [label="<string>"]
11    ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
17 }
```

تعریف ۲.۱.۳ > بلوک model

بلوک model نام کلاس تولیدشده، شناسه گره ورودی و شکل تنسور ورودی، و شناسه گره خروجی را مشخص می‌کند.

تعریف ۳.۱.۳ > بلوک graph

بلوک graph تمام گره‌های لایه و یال‌های جهت‌دار را تعریف می‌کند. ترتیب اعلان یال‌ها اهمیتی ندارد؛ کامپایلر ترتیب اجرا را از ساختار گراف محاسبه می‌کند.

تعریف ۴.۱.۳ > بلوک config

بلوک config مقادیری را برای بلوک __main__ کد تولیدشده تعیین می‌کند. batch_size پیش‌فرض ۱ و device پیش‌فرض 'cpu' است.

۲.۳ گرامر

تعریف ۱.۲.۳ > آمار گرامر g.NNGraph

- فایل گرامر: NNGraph.g4 (۵۶ خط)
- قوانین تجزیه‌گر: ۱۴ قانون (snake_case)
- توکن‌های واژه‌نگار: ۲۰ توکن (ALL_CAPS)

قوانین کامل گرامر:

قطعه کد ۲.۲.۳ > گرامر g.NNGraph

```

1 grammar NNGraph;
2 program      : model_block graph_block config_block? EOF;
3 model_block  : MODEL ID '{' input_decl output_decl '}';
4 input_decl   : INPUT ID ':' TENSOR shape_expr;
5 output_decl  : OUTPUT ID;
6 graph_block  : GRAPH '{' (node_decl | edge_decl)* '}';
7 node_decl    : NODE ID ':' layer_expr;
8 layer_expr   : ID '(' param_list? ')';
9 param_list   : param (',' param)*;
10 param       : ID '=' value;
```

```

11 edge_decl : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block: CONFIG '{' config_entry* '}';
13 config_entry: ID '=' value;
14 value : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15       | STRING | NONE | shape_expr;
16 shape_expr : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

نکته مهم طراحی: input_decl از shape_expr استفاده می‌کند نه TENSOR. زیرا ' (' shape_expr ') ' خود پرانتزها را در بر می‌گیرد. این خطای رایج در نسخه‌های اولیه گرامر برطرف شد.

۳.۳ سیستم انواع

مثال	نوع Python	نوع در DSL
128	int	int
1.0	float	float
false / true	bool	bool
"relu"	str	string
(64, 32, 32)	tuple[int]	shape
None	NoneType	None

تمایز بین int و float در زمان کامپایل بررسی می‌شود. مثلاً Dropout (p=1) خطا می‌دهد چون p انتظار float دارد نه int.

۴.۳ انواع لایه

۱.۴.۳ لایه‌های پارامتردار

این لایه‌ها در `__init__` به صورت `self.<id> = nn.X(...)` تولید می‌شوند.

کلیدواژه DSL	کلاس PyTorch	پارامترهای اصلی
Linear	<code>nn.Linear</code>	<code>bias, out_features, in_features</code>
Conv2d	<code>nn.Conv2d</code>	<code>padding, stride, kernel, out_ch, in_ch</code>
Conv1d	<code>nn.Conv1d</code>	<code>stride, kernel, out_ch, in_ch</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>stride, kernel</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>stride, kernel</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>end_dim, start_dim</code>
Embedding	<code>nn.Embedding</code>	<code>embedding_dim, num_embeddings</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>num_heads, embed_dim</code>
LSTM	<code>nn.LSTM</code>	<code>num_layers, hidden_size, input_size</code>
GRU	<code>nn.GRU</code>	<code>hidden_size, input_size</code>

۲.۴.۳ فعال‌سازی‌ها

.ELU(alpha), LeakyReLU(negative_slope), Softmax(dim), GELU, Tanh, Sigmoid, ReLU

۳.۴.۳ عملیات گراف (بدون ماژول)

این انواع در `__init__` خطی تولید نمی‌کنند:

کلیدواژه	پارامترها	کد تولیدشده
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a,b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int, dim: int</code>	<code>parts = torch.chunk(x,n,dim=d)</code>

۵.۳ نحو یال

تعریف ۱.۵.۳ یال ساده

```
edge src -> dst
```

یک یال جهت‌دار از گره src به گره dst تعریف می‌کند.

تعریف ۲.۵.۳ یال برچسب‌دار

```
edge src -> dst [label="name"]
```

یک یال با برچسب رشته‌ای تعریف می‌کند. برچسب‌ها دو نقش دارند:

- "shortcut": یال اتصال کوتاه به گره (Residual) را مشخص می‌کند.
- "residual_*": اتصال کوتاه به گره‌ای با دو ورودی (الگوی Transformer).

۶.۳ مثال کامل: MLP

فایل mlp.nng. برای یک شبکه سه‌لایه:

قطعه کد ۱.۶.۳ mlp.nng

```

1 model MLP {
2     input x : tensor(784)
3     output out
4 }
5
6 graph {
7     node fc1 : Linear(in_features=784, out_features=256)
8     node relu1 : ReLU()
9     node fc2 : Linear(in_features=256, out_features=128)
10    node relu2 : ReLU()
11    node drop : Dropout(p=0.3)
12    node fc3 : Linear(in_features=128, out_features=10)
13    node out : Softmax(dim=1)
14
15    edge x -> fc1
```

```

16     edge fc1    -> relu1
17     edge relu1 -> fc2
18     edge fc2    -> relu2
19     edge relu2 -> drop
20     edge drop   -> fc3
21     edge fc3    -> out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27 }

```

کد PyTorch تولیدشده:

قطعه کد ۲.۶.۳ > کد PyTorch تولیدشده برای MLP

```

1  import torch
2  import torch.nn as nn
3
4
5  class MLP(nn.Module):
6      def __init__(self):
7          super(MLP, self).__init__()
8          self.fc1 = nn.Linear(in_features=784,
9                               ↪ out_features=256)
9          self.relu1 = nn.ReLU()
10         self.fc2 = nn.Linear(in_features=256,
11                              ↪ out_features=128)
11         self.relu2 = nn.ReLU()
12         self.drop = nn.Dropout(p=3.0)
13         self.fc3 = nn.Linear(in_features=128,
14                              ↪ out_features=10)
14         self.out = nn.Softmax(dim=1)
15
16     def forward(self, x):

```

```
17         x = self.fc1(x)
18         x = self.relu1(x)
19         x = self.fc2(x)
20         x = self.relu2(x)
21         x = self.drop(x)
22         x = self.fc3(x)
23         x = self.out(x)
24         return x
25
26
27 if __name__ == '__main__':
28     device = torch.device('cuda')
29     model = MLP().to(device)
30     x = torch.randn(64, 784).to(device)
31     print(model(x).shape)
```

فصل ۴

پیاده‌سازی

۱.۴ معماری کامپایلر

تعریف ۱.۱.۴ > خط لوله کامپایل

کامپایلر چهار مرحله دارد:

۱. تجزیه: `FileStream` → `NNGraphLexer` → `NNGraphParser.program()`

درخت تجزیه

۲. تحلیل معنایی: `NNGraphCustomListener + ParseTreeWalker` → مدل گراف

۳. تولید کد: `CodeGenerator.generate()` → رشته Python

۴. خروجی: نوشتن به فایل `.py` یا چاپ در `stdout`

۱.۱.۴ مسئولیت فایل‌ها

فایل	نقش
NNGraph.g4	منبع حقیقت گرامر (۵۶ خط)
gen/NNGraphLexer.py	تولیدشده توسط FANTLR
gen/NNGraphParser.py	تولیدشده توسط FANTLR
gen/NNGraphListener.py	listener پایه – نباید ویرایش شود
custom_listener.py	تحلیل معنایی (۲۹۳ خط)
code_generator.py	تولید کد PyTorch (۲۳۹ خط)
main.py	رابط CLI و compile_nng() (۷۳ خط)

۲.۴ تحلیل‌گر معنایی

تعریف ۱.۲.۴>NNGraphCustomListener

کلاس NNGraphCustomListener(NNGraphListener) در custom_listener.py تمام منطق تحلیل معنایی را پیاده می‌کند. حالت داخلی:

```
dict[id, {type, params, line}]:self.nodes •
list[{src, dst, label, line}]:self.edges •
dict[key, value]:self.config •
```

۱.۲.۴ انتشار مقدار در درخت

متدهای exitShape_expr و exitValue مقادیر را روی خود context ذخیره می‌کنند:

قطعه کد ۲.۲.۴>NNGraphCustomListener در ctx.pvalue انتشار مقدار exitShape_expr

```
1 def exitShape_expr(self, ctx):
2     ints = [int(ctx.INT_LITERAL(i).getText())
3             for i in range(ctx.INT_LITERAL().__len__())]
4     ctx.pvalue = tuple(ints)
5     ctx.ptype = tuple
```

این رویکرد تضمین می‌کند که هر context پس از خروج از آن، مقادیر فرزندانش را در دسترس دارد—الگویی

که در پروژه EVM مرجع نیز استفاده شده است.

۲.۲.۴ قوانین اعتبارسنجی

هشت قانون اعتبارسنجی در دو فاز اجرا می‌شوند:

فاز اول (در زمان پیمایش درخت):

۱. شناسه تکراری گره: در `exitNode_decl`

۲. ارجاع نامعتبر در یال: در `exitEdge_decl`

۳. نوع لایه ناشناس: در `exitNode_decl`

۴. عدم تطابق نوع پارامتر: در `exitNode_decl`

فاز دوم (در `exitGraph_block`، پس از تکمیل گراف):

۵. گره خروجی اعلان‌نشده: در `_validate_output_declared`

۶. گره (هشدار): در `_validate_orphans`

۷. آریتی `Residual != ۲`: در `_validate_residual_arity`

۸. تشخیص دور با `Kahn`: در `_validate_no_cycles`

۹. قابلیت دسترسی `BFS`: در `_validate_reachability`

۳.۲.۴ قالب پیام خطا

قطعه کد ۳.۲.۴ > قالب خطا و هشدار

```
1 [line L:C] SemanticError: <message>
2 [line L] Warning: <message>
```

L شماره سطر (از ۱) و C ستون توکن (از ۰) است. هشدارها فقط در `stderr` چاپ می‌شوند و کامپایل را متوقف نمی‌کنند.

۴.۲.۴ `_FakeCtx` برای خطاهای پس از پیمایش

خطاهایی که در `exitGraph_block` رخ می‌دهند `context` زنده ندارند. یک کلاس داخلی `_FakeCtx` شماره سطر را از رکورد اعلان گره حمل می‌کند:

قطعه کد ۴.۲.۴ > کلاس FakeCtx برای حفظ شماره خط

```

1 class FakeCtx:
2     class start:
3         line = info["line"]
4         column = 0

```

۳.۴ مولد کد

تعریف ۱.۳.۴ > CodeGenerator

کلاس CodeGenerator در code_generator.py چهار گام داخلی دارد:

۱. `_compute_graph()`: ساخت لیست‌های مجاورت و مرتب‌سازی توپولوژیک
۲. `_assign_vars()`: تخصیص نام متغیر Python به هر گره
۳. `_emit_forward() + _emit_init()`: تولید خطوط کد
۴. `_assemble()`: ترکیب در سورس Python نهایی

۴.۴ الگوریتم تخصیص متغیر

تعریف ۱.۴.۴ > اصل اساسی تخصیص متغیر

`var_at[n] = 'x'` فقط وقتی که `out_degree[predecessor] == 1` وقتی یک گره به چند مقصد فرستاده می‌شود (`out_degree > 1`)، تنسور آن هنوز توسط شاخه‌های دیگر نیاز است. بازنویسی `x` در این نقطه آن را خراب می‌کند. به جای آن، گره بعدی نام شناسه خودش را به عنوان متغیر دریافت می‌کند.

چهار قانون کامل در `_assign_vars`:

قطعه کد ۲.۴.۴ > قوانین تخصیص متغیر در `_assign_vars`

```

1 Rule 1 - Input node:
2     var_at[input_id] = 'x'
3
4 Rule 2 - Merge node (in_degree > 1):

```

```

5     var_at[n] = 'x'          if out_degree[n] <= 1
6     var_at[n] = node_id     if out_degree[n] > 1
7
8 Rule 3 - Branch start (predecessor.out_degree > 1):
9     var_at[n] = node_id
10
11 Rule 4 - Linear continuation (predecessor.out_degree == 1):
12     var_at[n] = var_at[predecessor]
```

۱.۴.۴ ردیابی MLP (زنجیره خطی)

مثال ۳.۴.۴ ردیابی متغیر برای MLP

توپولوژی:

`x -> fc1 -> relu1 -> fc2 -> relu2 -> drop -> fc3 -> out`

هر predecessor دارای `out_degree == 1` است $\Rightarrow$ قانون ۴ همیشه اعمال می‌شود $\Rightarrow$ همه گره‌ها `var_at = 'x'` دارند. کد forward از `x` در همه جا استفاده می‌کند.

۲.۴.۴ ردیابی ResBlock (فناوت سپس ادغام)

مثال ۴.۴.۴ ردیابی متغیر برای ResBlock

`x` به `conv1` و `skip` (اتصال کوتاه) فرستاده می‌شود. پس `out_degree[x] = 2`.
`conv1`: قانون ۳ $\Rightarrow$ `var_at = 'conv1'`
`bn1`, `relu1`, `conv2`, `bn2`: قانون ۴ $\Rightarrow$ همه `'conv1'`
`var_at = 'x'` $\Rightarrow$ `out_degree=1`, `in_degree=2` (Residual, `skip`): قانون ۲،
 کد تولیدشده:

قطعه کد ۵.۴.۴ › pass forward تولیدشده برای ResBlock

```

1 conv1 = self.conv1(x)
2 conv1 = self.bn1(conv1)
3 conv1 = self.relu1(conv1)
4 conv1 = self.conv2(conv1)
5 conv1 = self.bn2(conv1)
6 x      = conv1 + x      # Residual node
7 x      = self.flatten(x)

```

۳.۴.۴ ردیابی InceptionBlock (دو شاخه موازی)

مثال ۶.۴.۴ › ردیابی متغیر برای InceptionBlock

$out_degree[x] = 2$ (به convA1 و convB1).
 • convA1: قانون ۳ $\Rightarrow$ 'convA1'
 • convB1: قانون ۳ $\Rightarrow$ 'convB1'
 • convA2: قانون ۴ $\Rightarrow$ ارث از 'convA1'
 • convB2: قانون ۴ $\Rightarrow$ ارث از 'convB1'
 • (Concat, cat1): $in_degree=2$: قانون ۲ $\Rightarrow$ 'x'
 کد تولیدشده:

قطعه کد ۷.۴.۴ › pass forward تولیدشده برای InceptionBlock

```

1 convA1 = self.convA1(x)
2 convA1 = self.convA2(convA1)
3 convB1 = self.convB1(x)
4 convB1 = self.convB2(convB1)
5 x      = torch.cat([convA1, convB1], dim=1) # Concat
        ↪ node
6 x      = self.bn(x)
7 x      = self.out(x)

```

۴.۴.۴ ردیابی TransformerEncoder (دو residual ضمنی)

مثال ۸.۴.۴ ردیابی متغیر برای TransformerEncoder

`.out_degree[norm1] = 2` و `out_degree[x] = 2`
 • `attn`: قانون ۳ $\Rightarrow$ `'attn'`
 • `drop1`: قانون ۴ $\Rightarrow$ `'attn'`
 • `norm1`: قانون ۲ $\Rightarrow$ `'norm1'` (`in_deg=2, out_deg=2`)
 • `ff1`: قانون ۳ $\Rightarrow$ `'ff1'`
 • `gelu1, drop2, ff2`: قانون ۴ $\Rightarrow$ `'ff1'`
 • `norm2`: قانون ۲ $\Rightarrow$ `'x'` (`in_deg=2, out_deg=1`)
 کد تولیدشده:

قطعه کد ۹.۴.۴ pass forward تولیدشده برای TransformerEncoder

```

1  attn, _ = self.attn(x, x, x)      # MultiHeadAttn
   ↪ triple-pass
2  attn      = self.drop1(attn)
3  norm1     = self.norm1(x + attn)  # implicit residual_1
4  ff1       = self.ff1(norm1)
5  ff1       = self.gelu1(ff1)
6  ff1       = self.drop2(ff1)
7  ff1       = self.ff2(ff1)
8  x         = self.norm2(norm1 + ff1) # implicit
   ↪ residual_2
9  x         = self.out(x)
10 x         = self.flatten(x)
11 return x

```

۵.۴.۴ موارد خاص تولید کد

الگو	شرط	کد تولیدشده
MultiHeadAttn	نوع لایه	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	نوع لایه	<code>out, _ = self.lstm(x)</code>
Residual	بدون ماژول	<code>out = main + shortcut</code>
Add	بدون ماژول	<code>out = a + b</code>
Concat	بدون ماژول	<code>out = torch.cat([a,b], dim=d)</code>
Split	بدون ماژول	<code>parts = torch.chunk(x,n,dim=d)</code>
Residual ضمنی	<code>label residual_*</code>	<code>out = self.norm(skip + main)</code>

نگاشت نام پارامتر: DSL از نام‌های کوتاه‌تر استفاده می‌کند. مولد کد آن‌ها را به نام‌های PyTorch نگاشت می‌کند: `in_ch → in_channels, out_ch → out_channels, kernel_size → kernel`

۶.۴.۴ ساختار کد خروجی

قطعه کد ۱۰.۴.۴ الگوی کد خروجی تولیدشده

```

1 import torch
2 import torch.nn as nn
3
4
5 class <ModelName>(nn.Module):
6     def __init__(self):
7         super(<ModelName>, self).__init__()
8         # one line per parameterized layer node
9
10    def forward(self, x):
11        # one line per node in topological order
12        return <output_var>
13
14
```

```
15 if __name__ == '__main__':  
16     device = torch.device('<device>')  
17     model = <ModelName>().to(device)  
18     x      = torch.randn(<batch_size>,  
19         ↪ <input_shape>).to(device)  
    print(model(x).shape)
```


فصل ۵

ارزیابی

۱.۵ مجموعه آزمایش

تعریف ۱.۱.۵ • آمار آزمایش

- تعداد کل تست‌ها: ۵۵
- فریم‌ورک: pytest نسخه ۲.۴.۸
- زمان اجرا: حدود ۹.۱ ثانیه

فایل	تست‌ها	پوشش
test_parser.py	۸	گرامر بدون خطای نحوی برای ۴ نمونه؛ config، پرچسب، شکل
test_listener.py	۱۴	۴ ورودی معتبر؛ ۱۰ شرط خطا با SystemExit
test_codegen.py	۱۹	نام کلاس‌های nn، مقادیر پارامتر، الگوهای forward
test_e2e.py	۱۴	کامپایل کامل + ایجاد نمونه + for- pass ward با شکل صحیح

دستور اجرای آزمایش‌ها:

قطعه کد ۲.۱.۵ > اجرای مجموعه آزمایش

```
1 python -m pytest tests/ -v
```

۲.۵ نتایج pass forward

مثال ۱.۲.۵ > MLP

- ورودی: $\text{tensor}(784)$ – خروجی: $(\text{batch}, 10)$
- آزمایش: Softmax ردیف‌ها به ۱ جمع می‌شوند ($\text{atol}=1\text{e}-5$)

مثال ۲.۲.۵ > ResBlock

- ورودی: $\text{tensor}(64, 32, 32)$ – خروجی: $(\text{batch}, 65536)$
- آزمایش: شکل صحیح؛ در حالت eval خروجی قطعی است

مثال ۳.۲.۵ > InceptionBlock

- ورودی: $\text{tensor}(32, 56, 56)$ – خروجی: $(\text{batch}, 48, 56, 56)$
- شاخه A: ۱۶ کانال؛ شاخه B: ۳۲ کانال؛ ادغام با $\text{Concat}(\text{dim}=1)$ → ۴۸ کانال

مثال ۴.۲.۵ > TransformerEncoder

- ورودی: $\text{tensor}(512, 128)$ – خروجی: $(1024, 128)$
- دو زیرلایه residual ضمنی بدون گره $\text{Residual}()$
- MultiHeadAttn با $\text{embed_dim}=128, \text{num_heads}=8,$ $\text{batch_first}=\text{True}$

۳.۵ اثبات درستی متغیرها

تعریف ۱.۳.۵ > اثبات عدم بازنویسی تنسور

- برای هر نمونه شاخه‌دار، $\text{data_ptr}()$ ورودی در طول محاسبات forward ردیابی شد:
- **ResBlock**: $\text{data_ptr}()$ در نقطه $x = \text{conv1} + x$ تغییر نکرده
 - **Inception**: $\text{data_ptr}()$ هنگام $\text{convA1}(x)$ و $\text{convB1}(x)$ یکسان است

• **Transformer**: `x.data_ptr()` هنگام `norm1(x + attn)`
`norm1.data_ptr()` هنگام `norm2(norm1 + ff1)` تغییر نکرده
 این نتیجه مستقیماً از اصل `var_at[n]='x'` فقط وقتی `out_degree=1` پیروی می‌کند.

۴.۵ آمار کدبیس

خطوط	فایل/دایرکتوری
۵۶	NNGraph.g4
۲۹۳	custom_listener.py
۲۳۹	code_generator.py
۷۳	main.py
۵۷۵	tests/ (۴ فایل)
۱۰۰≈	required_code_collection/
۱۳۳۶≈	جمع

- زبان: Python ۳.۱۴.۳
- ANTLR: ۲.۱۳.۴
- PyTorch: ۰.۱۰.۲
- pytest: ۲.۴.۸

فصل ۶

تصمیمات طراحی

۱.۶ Visitor در برابر Listener

الگوی Listener به صورت طبیعی با انباشت حالت سازگار است. متدهای `exit*` می‌توانند مستقیماً روی `self.nodes` و `self.edges` عمل کنند بدون اینکه مقادیر را از طریق درخت برگردانند.

۲.۶ `load_from_listener()` در برابر `generate(traversal)`

پروژه EVM مرجع از `CodeGenerator.generate(traversal)` استفاده می‌کند که یک لیست توکن پس‌ترتیب دریافت می‌کند—مناسب برای تولید بایت‌کد پشت‌های. برای کامپایلر گراف، ورودی طبیعی ساختار گراف (گره‌ها + یال‌ها) است نه جریان خطی توکن. پس `load_from_listener(listener)` مستقیم‌تر است.

۳.۶ نام‌گذاری متغیر با `node_id`

مثال‌های مشخصات زبان از نام‌های تک‌حرفی یا توصیفی استفاده می‌کنند. این کامپایلر از شناسه واقعی گره استفاده می‌کند زیرا:

- قطعی است و برای گراف‌های بزرگ‌تر بدون شمارنده کار می‌کند
- خوانا است—متغیر `convA1` واضح‌تر از `a` است
- خروجی ریاضی یکسان است

فصل ۷

نتیجه‌گیری و کارهای آینده

۱.۷ نتیجه‌گیری

NNGraph DSL یک زنجیره کامپایلر چهار مرحله‌ای از `nng` به `nn.Module` قابل اجرا ارائه می‌دهد. کامپایلر گرامر ANTLR۴ با ۱۴ قانون تجزیه‌گر، ۲۴ نوع لایه، ۸ قانون اعتبارسنجی، و الگوریتم تخصیص متغیر که شاخه‌بندی پیچیده را بدون بازنویسی تنسور مدیریت می‌کند را ارائه می‌دهد. کامپایلر با ۵۵ تست در چهار معماری نمونه اعتبارسنجی شده است.

۲.۷ کارهای آینده

قابلیت	توضیح
استنتاج شکل	انتشار شکل تنسور و تشخیص عدم تطابق در کامپایلر
صادرکردن ONNX	تولید گراف سازگار با ONNX به جای Python
داربست آموزش	تولید حلقه آموزش با بهینه‌ساز و تابع خطا از <code>config</code>
تجسم Graphviz	خروجی نمودار گراف مدل
زیرگراف‌های نام‌دار	بلوک‌های ماکرو قابل استفاده مجدد
کنترل جریان پویا	شاخه‌های <code>else/if</code> درون <code>forward</code>
کوانتیزاسیون	متادیتا برای تبدیل مدل به نسخه‌های کوانتیزه

منابع

۱. Parr, T. (۲۰۱۳.) The Definitive ANTLR ۴ Reference. Pragmatic Bookshelf.
۲. Paszke, A., et al. (۲۰۱۹.) PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS ۲۰۱۹.
۳. NADER Framework: Neural Architecture Design via Representation (referenced in NNGraph_DSL_Specification.md.)
- Zhang, A. M. amznotes LaTeX template. github.com/alexmingzhang/latex-notes-template